Project 2: 2D Search

Joel Batac Joshua Gilo

Technical Panel Presentation

March 8, 2026

Presentation Roadmap

1. Context and Objectives
2. Methods and Algorithm Mechanics
3. Random Growth Results
4. QR Search Results
5. Contrived Worst-Case Results
6. Cross-Section Synthesis
7. Max-Input Head-to-Head
8. Conclusions and Next Steps

Working Hypotheses

RQ1. Which algorithm minimizes equality comparisons per scenario?
RQ2. How does wall-clock trend with comparison results?
RQ3. How do results change from average workloads to adversarial layouts?

Hypothesis

- Hash/skip-heavy methods **should** reduce comparisons.
- Wall-clock can diverge due to implementation details (NumPy/Python/automata overhead), so it is secondary evidence.

For this, **comparisons are primary**; wall-clock is retained as secondary runtime evidence.

Study Scope at a Glance

Algorithms	Naive, Rabin-Karp, Vectorized RK, Greedy KMP, Greedy BM, Bird-KMP, Bird-BM
Random section size	980 rows total
QR section size	420 rows over 60 QR images
Contrived section size	49 rows
Head-to-head size	5 rows (5 algorithms)

Algorithm Family Overview

Naive	Basic 2D sliding-window exact matching baseline.
Rabin-Karp	2D rolling hash over windows, then exact verification [1].
Vectorized RK	Same RK logic with NumPy-wide window hashing.
Greedy KMP	KMP filter on first row, then greedy full-row verification [2].
Greedy BM	Bad-character BM filter on first row, then greedy verification [3].
Bird-KMP	Column-encoding + Aho-Corasick-like column scan + KMP over IDs [4, 5].
Bird-BM	Same Bird preprocessing, BM over column-ID sequence.

What We Measured

- Every algorithm returns `(comparison_count, match_position)`.
- `Counter` wraps equality checks; this is the primary metric used in analysis.
- Wall-clock is captured externally per run as secondary runtime evidence.

Interpretation rule

comparison_count is primary (algorithm behavior), while **wall-clock** is secondary (implementation runtime evidence).

Naive Search: Core Mechanics

Input: text $T \in \{0,1\}^{N \times N}$, pattern $P \in \{0,1\}^{M \times M}$

1. Enumerate every valid top-left window (i, j).
2. Compare all M^2 pattern cells against $T[i : i + M, j : j + M]$.
3. Stop at first full match, else return `None`.

Behavior of Naive Search

- Observed Strength: very transparent baseline and stable implementation.
- Observed Weakness: no pruning; cost grows quickly when no early match occurs.
- Implementation details: minimal overhead makes it a useful control for interpreting other methods.

2D Rabin-Karp: Rolling Hash Pipeline

1. Compute pattern hash using row and column polynomial bases.
2. Build row-wise rolling hashes for text.
3. Build vertical rolling hashes over those row hashes.
4. On hash hit, perform exact cell verification.

Why it helps

Most windows are rejected cheaply by hash mismatch before full comparison.

Vectorized RK

- Uses NumPy sliding windows and reduction operations to hash many windows together.
- Reduces Python-loop overhead in hash computation stage.

Expected effect

Lower runtime than loop RK when vectorized operations amortize setup overhead.

Greedy KMP

1. Build LPS array from first pattern row.
2. Run KMP across each candidate top text row.
3. For first-row matches, greedily verify remaining rows directly.

- Effective when first-row mismatches are frequent.
- Can overwork when first row matches often but full pattern fails.

Greedy BM

1. Build bad-character table from first pattern row.
2. Match right-to-left on that row; shift aggressively on mismatch.
3. If first-row match, verify remaining rows greedily.

Observed Implementation Effect

Greedy BM benefits when first-row mismatches are frequent (random-like data), but degrades when many top rows trigger verification.

Bird Methods

1. Encode pattern columns into IDs.
2. Build automaton over unique column signatures.
3. For each top-row offset, derive a column-ID sequence of text windows.
4. Run 1D matcher over IDs: KMP variant or BM variant.

Goal

Reduce expensive full-grid matching to structured column-sequence matching.

Bird-KMP and Bird-BM: Tradeoffs

- Usually excellent in comparison-count efficiency (especially Bird-BM).
- Preprocessing and automaton traversal add heavy constant-time overhead.
- This overhead becomes non-negligible as text/pattern sizes increase.
- In our profiling, `_build_C_row` and `_ac_next` dominate runtime.

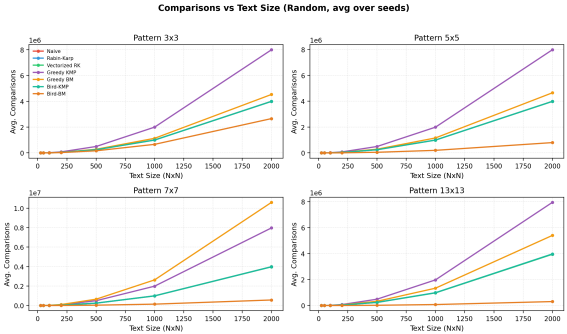
Why Comparison and Runtime Rankings Diverge

Counted work	Equality checks routed through `Counter`.
Non-counted work	Hash updates, dict lookups, automaton transitions, NumPy reductions, memory traversal.
Outcome	Algorithms with fewer counted comparisons can still be slower overall.

Experimental Sections and What They Stress

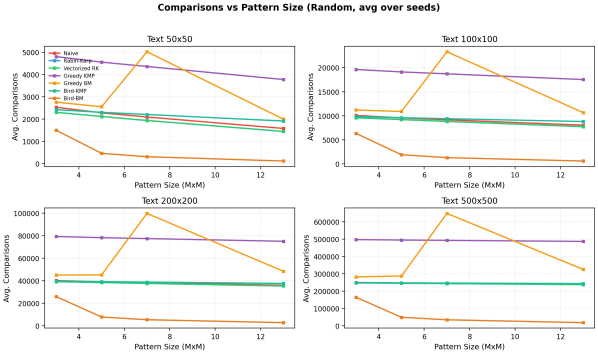
Random Growth	Scaling behavior under structured no-match conditions in notebook generation.
QR Search	Realistic pattern detection over downsampled QR-derived matrices.
Contrived	Near-worst-case scanning with guaranteed late match.
Head-to-Head	Maximum-size stress test revealing constant-factor realities.

Random Suite: Bird-BM Minimizes Comparison Volume



Best average comparison count: **Bird-BM** (204743.7).

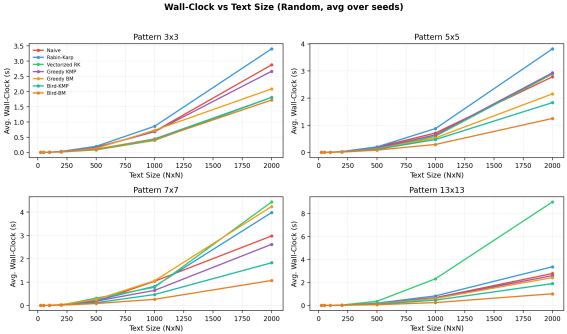
Pattern-Size Sensitivity Differs by Algorithm Family



Interpretation

Bird-BM remains comparison-efficient as pattern size increases, while first-row-greedy variants scale less favorably in this suite.

Runtime Trend on Random Inputs

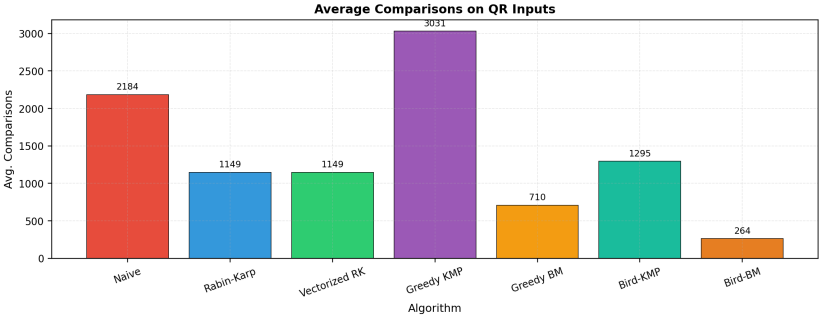


Winner (Runtime): Bird-BM (0.237957 s).

Random Section Takeaway Table

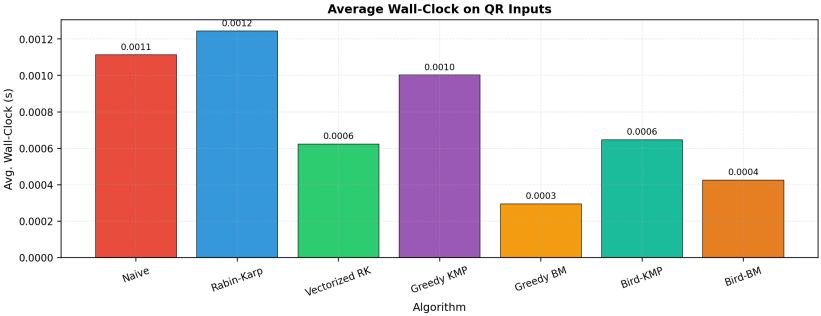
Algorithm	Mean Comparisons	Mean Time (s)
Bird-BM	204743.707	0.237957
Rabin-Karp	750996.429	0.675892
Vectorized RK	750996.429	0.828940
Naive	753130.957	0.548849
Bird-KMP	754329.014	0.348045
Greedy BM	1189064.686	0.521715
Greedy KMP	1508161.414	0.508494

On QR Data, Bird-BM Uses the Fewest Comparisons



Best average comparison count: **Bird-BM** (263.6).

Runtime on QR Inputs



Winner (Runtime): Greedy BM (0.000295 s).

Per-Image Stability Across 60 QR Inputs

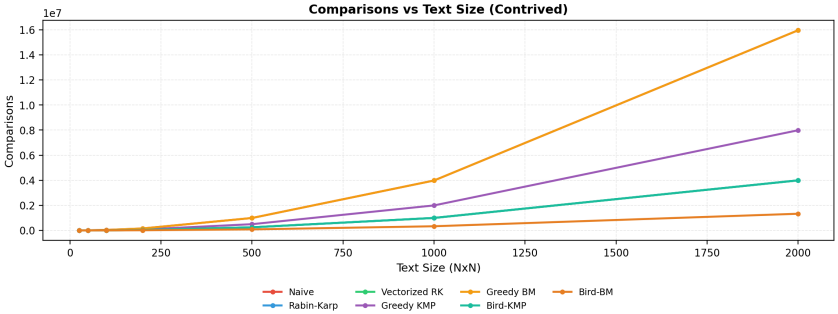
Observation

Bird-BM remains the lowest-comparison method across images; runtime ordering is presented as supporting evidence.

QR Section: Corrected Numeric Callouts

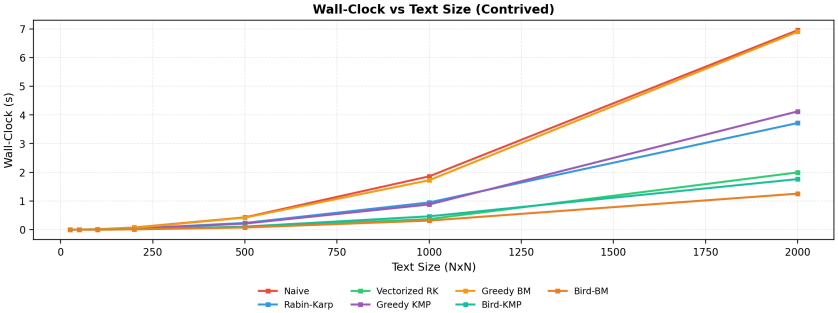
- Mean comparisons (Rabin-Karp): **1148.933**.
- Mean comparisons (Vectorized RK): **1148.933**.
- Mean comparisons (Bird-BM): **263.633**.
- Winner (Runtime): **Greedy BM** (0.000295 s).

Contrived Case: Bird-BM Retains the Lowest Comparisons



Best mean comparisons: Bird-BM (251666.7), while Greedy BM rises to 3019314.714.

Runtime Under Adversarial Placement



Winner (Runtime): Bird-BM (0.238081 s).

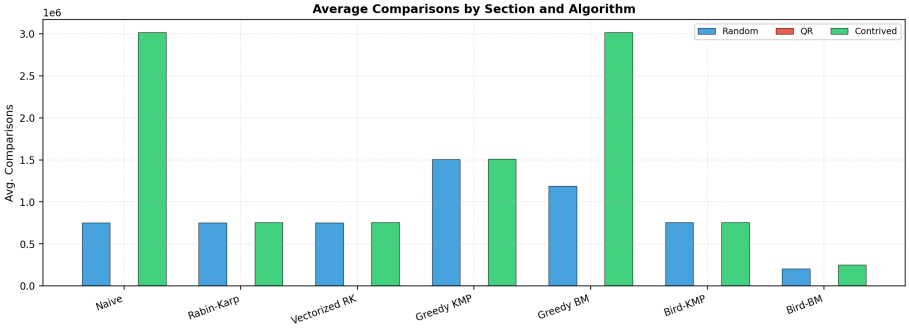
Contrived Section Summary

Algorithm	Mean Comparisons	Mean Time (s)
Bird-BM	251666.714	0.238081
Greedy BM	3019314.714	1.304504
Vectorized RK	754836.429	0.353938
Naive	3019314.714	1.334626

Interpretation

In this contrived setup, frequent first-row triggers increase verification work; Bird-BM still keeps comparisons low, while Greedy BM degrades.

Cross-Section Comparison Snapshot



Observation

Bird-BM repeatedly minimizes comparisons across sections wall-clock is retained as secondary runtime evidence

Metric Interpretation Matrix

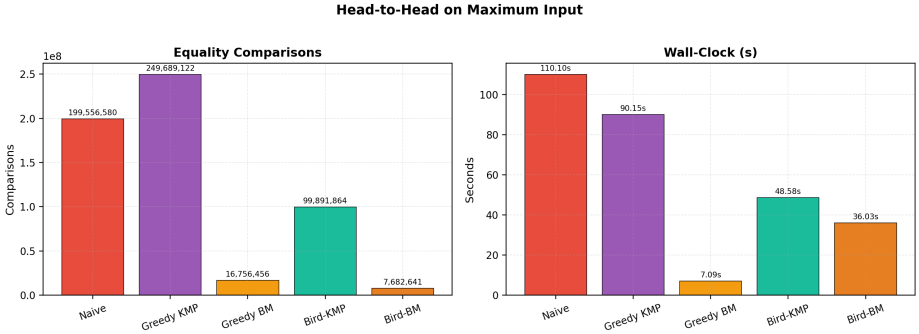
Metric	Strength	Limitation
Comparisons	Shows filtering efficiency and candidate-pruning behavior	Misses overhead from hashing/automata/memory traversal
Wall-clock (secondary)	Captures end-to-end implementation behavior	Hardware/environment dependent; secondary for algorithm ranking

Primary lens: comparisons. Secondary lens: wall-clock runtime evidence before deployment.

Scenario-Based Algorithm Selection (Comparisons First)

Scenario	Primary Pick (Comparisons)	Winner (Runtime)
Random-growth style runs	Bird-BM	Bird-BM
QR-like real workload	Bird-BM	Greedy BM
Contrived adversarial placement	Bird-BM	Bird-BM

10000x10000 Stress Test: Comparisons and Runtime



Winner (Comparisons): Bird-BM (7682641) | **Winner (Runtime):** Greedy BM (7.0900 s).

Why Bird-BM Can Win Comparisons but Lose Runtime

- In this implementation, Bird-family preprocessing and traversal overhead is significant.
- At larger texts/patterns, that overhead becomes non-negligible.
- Result: Bird-BM has lowest comparisons (7682641) but higher runtime (36.0266 s).
- Greedy BM remains faster in runtime terms (7.0900 s) despite higher comparisons.

Final Conclusions

1. **Comparison leader:** Bird-BM is consistently best or near-best in comparison count.
2. **Wall-clock is secondary:** we report it as runtime evidence, not as the primary grading metric.
3. **Metric divergence is real:** implementation overhead changes wall-clock without changing comparison logic.
4. **Decision rule:** pick algorithm by workload and objective, not by one global metric.

References I

R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987.

D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

R. S. Boyer and J. S. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

R. S. Bird, "Two-dimensional pattern matching," *Information Processing Letters*, vol. 6, no. 5, pp. 168–170, 1977.

A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.

Backup: Random Means (All Algorithms)

Algorithm	Comparisons	Wall-clock (s)
Bird-BM	204743.707	0.237957
Bird-KMP	754329.014	0.348045
Greedy BM	1189064.686	0.521715
Greedy KMP	1508161.414	0.508494
Naive	753130.957	0.548849
Rabin-Karp	750996.429	0.675892
Vectorized RK	750996.429	0.828940

Backup: QR Means (All Algorithms)

Algorithm	Comparisons	Wall-clock (s)
Bird-BM	263.633	0.000426
Bird-KMP	1294.600	0.000647
Greedy BM	710.017	0.000295
Greedy KMP	3030.550	0.001002
Naive	2184.150	0.001113
Rabin-Karp	1148.933	0.001244
Vectorized RK	1148.933	0.000623

Backup: Contrived Means (All Algorithms)

Algorithm	Comparisons	Wall-clock (s)
Bird-BM	251666.714	0.238081
Bird-KMP	755930.571	0.337132
Greedy BM	3019314.714	1.304504
Greedy KMP	1511864.143	0.751434
Naive	3019314.714	1.334626
Rabin-Karp	755387.000	0.708536
Vectorized RK	754836.429	0.353938